

Security Risks in AI Infrastructure

Threats across data, models, code, and compute



Why AI Security Is Different

Traditional security approaches fall short when protecting AI systems

Expanded Attack Surface

Complex journey from **data** → **features** → **models** → **inference** → **pipelines** creates multiple vulnerable points

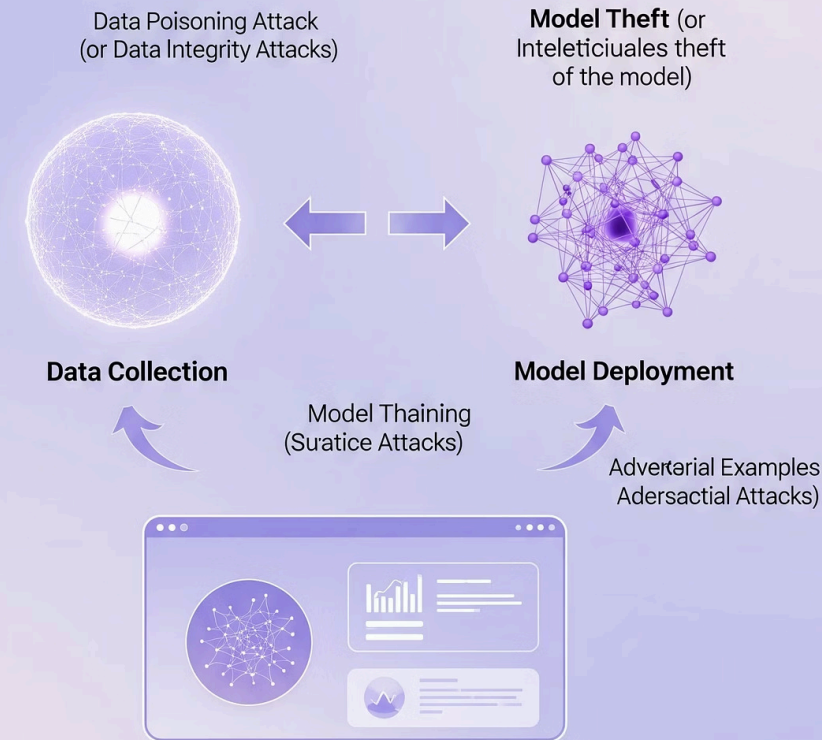
Infrastructure Complexity

GPU clusters & specialized runtimes add new security challenges beyond standard compute environments

Dual-Nature Threats

Models can be **both target & weapon** (e.g., jailbreaks to bypass safety guardrails, data exfiltration via model outputs)

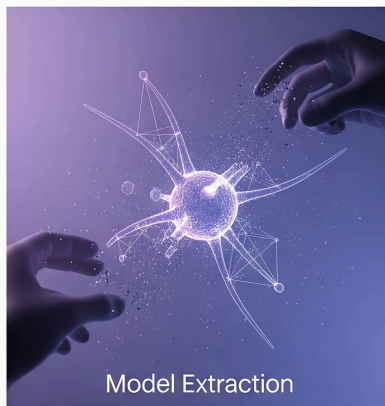
Expanded AI Security Attack Surface



Threat Landscape (At a Glance)



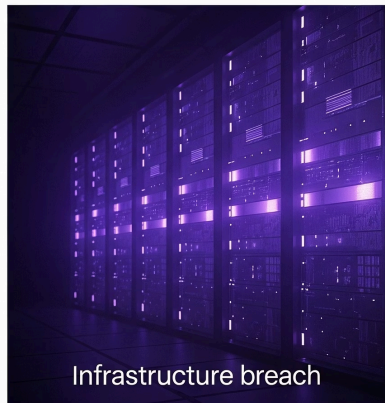
Data Theft



Model Extraction



Malicious Prompts



Infrastructure breach

Data

Theft, leakage, poisoning attacks that compromise training integrity

Model

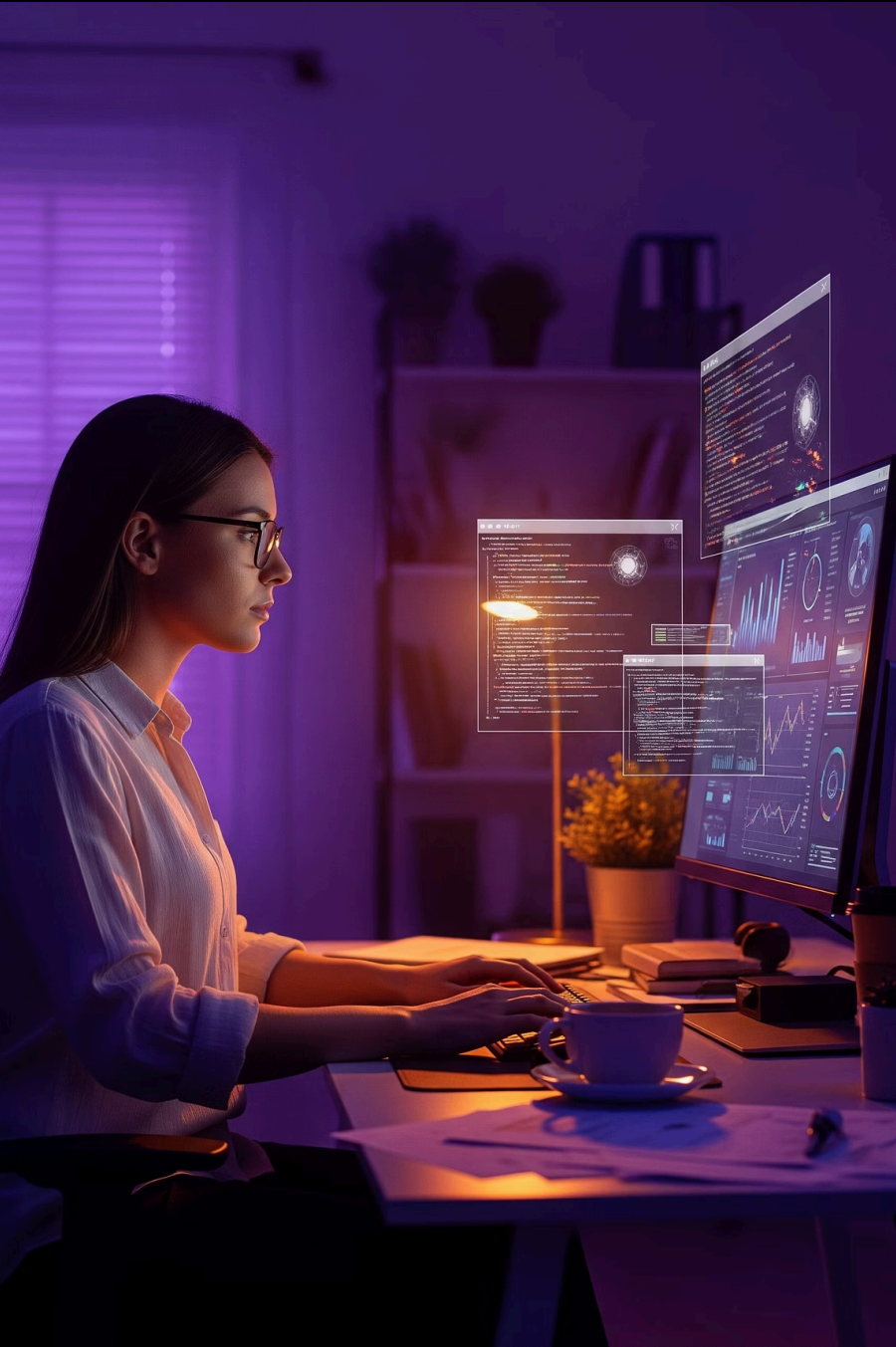
Theft/extraction, inversion, membership inference attacks exposing sensitive information

Inference

Prompt injection, adversarial inputs, DoS attacks targeting runtime behavior

Infra

Supply-chain vulnerabilities, secrets exposure, lateral movement within systems



Data-Centric Risks

The foundation of AI systems is their most vulnerable component

Training Data Poisoning

Attackers manipulate datasets through label flips and backdoor insertions, creating models with hidden vulnerabilities

Shadow Datasets & PII Sprawl

Untracked copies in S3/object stores lead to unmanaged sensitive data and potential regulatory violations

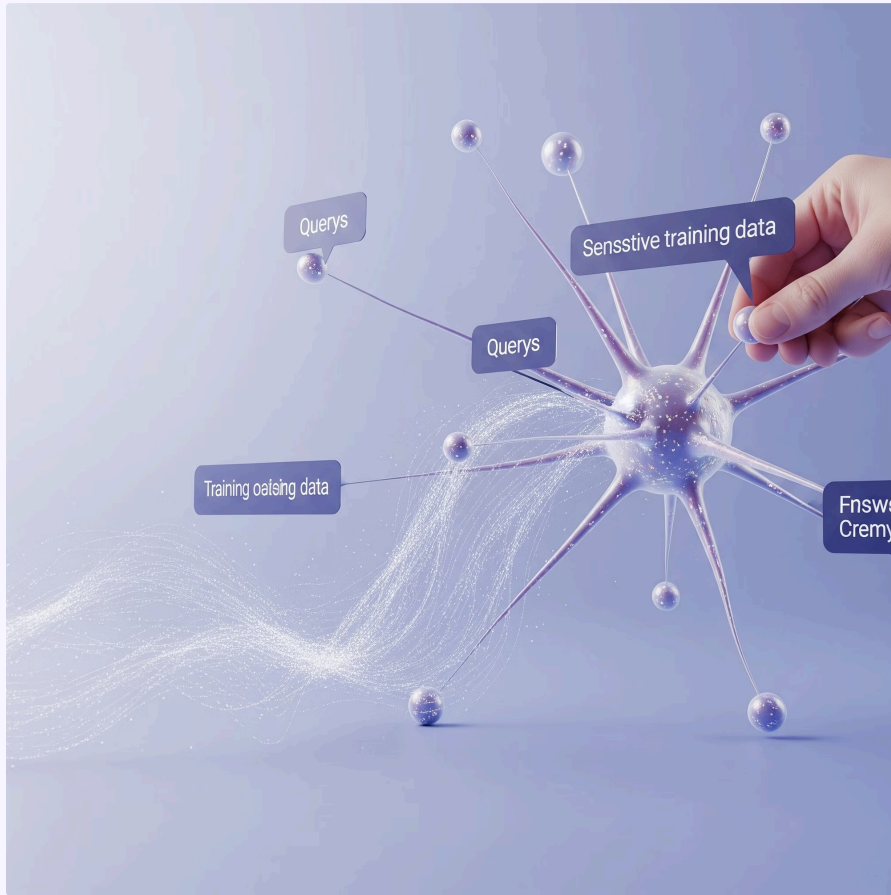
Weak Access Controls

Insufficient protections on feature stores & backups allow unauthorized access to training data

Linkage Attacks

Quasi-identifiers in datasets enable reconstruction of sensitive information even from "anonymized" data

Model-Centric Risks



1 Model Extraction

Attackers harvest responses from API to recreate proprietary models, stealing intellectual property

2 Model Inversion

Recover sensitive training samples by exploiting the model's memory of training data

3 Membership Inference

Detect if specific users' data was in the training set, violating privacy expectations

4 Watermark/Backdoor Triggers

Hidden patterns embedded in weights can be activated by specific inputs to force malicious behaviors



Inference-Time Risks



Prompt Injection / Jailbreaks

Crafted inputs that bypass safety guardrails in LLMs, enabling policy violations or system manipulation



Adversarial Examples

Subtly modified inputs that cause vision/speech models to make dangerous misclassifications



Tool/Plugin Abuse

Exploiting connected tools to access unauthorized resources or exfiltrate secrets



DoS Attacks

Hot-path overload causing latency SLO breaches and service degradation

Pipeline & Supply Chain

Vulnerabilities in the development lifecycle

- **Malicious Model Artifacts**

Unsigned container images and model files can contain backdoors or credential stealers

- **Compromised Training Scripts**

CI runners executing untrusted code during the model development process

- **Dependency Attacks**

PyPI/NPM **typosquatting** introduces malicious code through package confusion

- **SBOM Gaps**

Unknown vulnerable components lurking in dependencies due to incomplete tracking





Cloud & Cluster Risks

IAM Vulnerabilities

Over-broad IAM roles and stale service accounts granting excessive privileges to attackers

Cloud Metadata Exposure

Instance metadata theft and misconfigured VPC egress allowing credential exfiltration

Kubernetes Weaknesses

Privilege escalation, pod escape, and weak NetworkPolicies enabling lateral movement

GPU Security Gaps

GPU scheduler abuse through noisy neighbor attacks and VRAM scraping

Multi-Tenancy & Data Isolation



Critical Isolation Failures

- Cross-tenant data access via shared **feature stores/vector DBs** exposing confidential information
- Inference servers hosting many models creating **noisy neighbors** that can leak information
- Inadequate **namespace/RBAC controls** allowing tenant boundary violations
- Missing per-tenant resource quotas enabling DoS against shared services
- Side-channel attacks against shared GPUs extracting model parameters

⚠ Multi-tenant AI infrastructures require additional isolation beyond standard cloud security practices

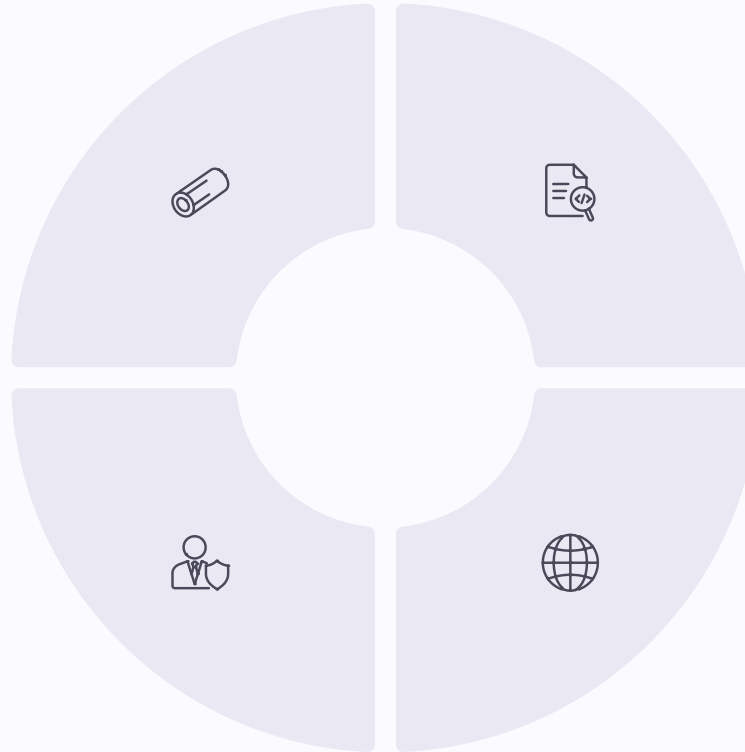
Governance & Compliance Gaps

Missing Audit Trails

No comprehensive logging of model changes & access patterns for forensics

Key Management Issues

Poor secret rotation practices and inadequate encryption key lifecycle



No Data Lineage

Cannot trace from source → features → model, creating compliance blind spots

Sovereignty Violations

Region/sovereignty issues from cross-border data storage and processing

These gaps create significant regulatory risk beyond the technical security vulnerabilities

Indicators of Compromise (IoCs)

Early warning signs your AI infrastructure has been breached



Unusual Inputs

Spikes in atypical prompts or token usage patterns indicating probing attacks

Performance Anomalies

Unexpected latency/throughput changes on inference endpoints

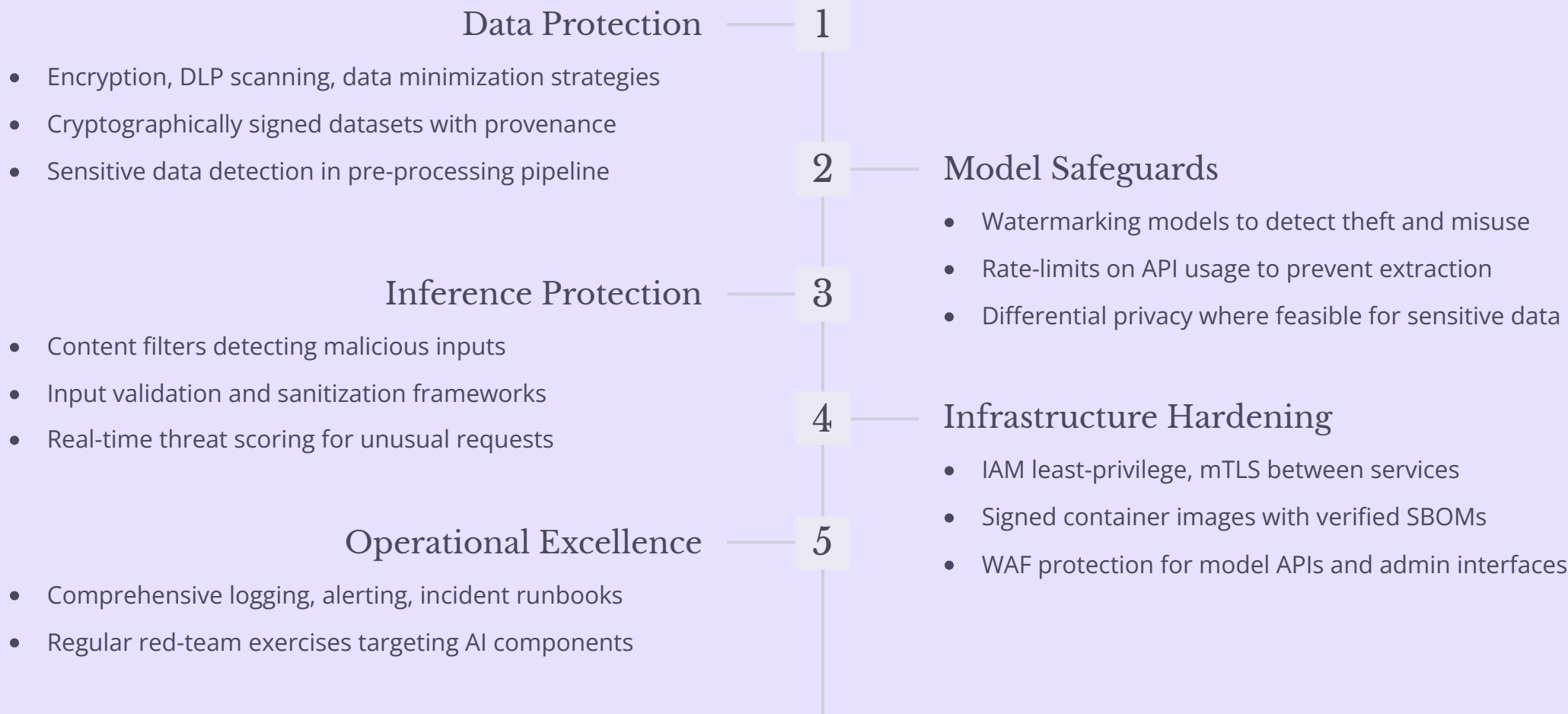
Container/Model Integrity

Unrecognized container digests or model hashes suggesting tampering

Access Patterns

Requests from unexpected principals/regions or outside normal hours

Mitigation Playbook (Preview)





Design Principles



Zero Trust by Default

Never trust, always verify across both network and identity boundaries



Secure-by-Default

Hardened templates for pipelines & serving infrastructure



Defense in Depth

Multiple security layers across data→model→API→cluster stack



Measure & Test

Regular chaos engineering + security drills for AI systems

Key Takeaways

1

New Attack Vectors

AI introduces **unique attack paths**—treat models like critical software assets requiring specialized protection

2

Holistic Protection

Secure **data + models + pipelines + clusters** as an integrated system rather than individual components

3

Foundational Security

Bake in **identity, isolation, and observability** from day one of your AI infrastructure design

4

Proactive Approach

Set up **guardrails now** to avoid costly security incidents that could damage both systems and reputation

Questions? Contact the Security Architecture team at security-architecture@company.com